Labo 07 — Réponses commentées

Chaque réponse suit le même schéma : la réponse, le mécanisme, la nuance ou le piège, un exemple vérifiable au terminal.

Question 1 — `localhost:5432` dans l'API

Réponse. Dans le conteneur `api`, `localhost` désigne **le conteneur `api`** : personne n'y écoute sur 5432, d'où `Connection refused`. Dans le conteneur `db`, `localhost:5432` est bien PostgreSQL — mais l'API n'y est pas. Correction : `SPRING_DATASOURCE_URL=jdbc:postgresql://db:5432/app`, le nom du conteneur, résolu par le DNS du réseau user-defined.

Pourquoi. Chaque conteneur a son network namespace, donc son propre `lo`. `127.0.0.1` ne traverse jamais une frontière de namespace.

Nuance. `pg_isready` dans `db` prouve que la base écoute, pas qu'elle est joignable depuis `api`. Le test pertinent se fait **depuis** `api` (ou depuis un conteneur jetable du même réseau).

Exemple.

```
podman run --rm --network stack alpine nc -zv db 5432          # db (10.89.0.2:5432) open
podman run --rm --network stack alpine nc -zv localhost 5432   # Connection refused
```

Question 2 — `IPAddress` vide

Réponse. Le conteneur est sur le réseau **par défaut rootless**, géré par `pasta`, qui copie l'adresse de l'hôte dans le conteneur au lieu de lui en attribuer une : Podman n'a donc pas d'« IP de conteneur » à afficher. Pour obtenir une adresse `10.88.0.x`, lancer le conteneur avec `--network podman` (le bridge), ou mieux, sur un réseau créé par vous.

Pourquoi. Un utilisateur ordinaire ne peut pas créer de bridge ni de NAT ; `pasta` fait tout en espace utilisateur, en reproduisant la pile de l'hôte. Les réseaux bridge rootless, eux, vivent dans un namespace privé de Podman.

Nuance. `HostConfig.NetworkMode` vaut `pasta` dans le premier cas, `bridge` dans le second. Docker root affiche toujours une IP, car son réseau par défaut est déjà un bridge.

Exemple.

```
podman run -d --name s alpine sleep 1d && podman inspect s --format
'{{.NetworkSettings.IPAddress}}|{{.HostConfig.NetworkMode}}'   # |pasta
podman run -d --name p --network podman alpine sleep 1d && podman inspect p --format
'{{.NetworkSettings.Networks.podman.IPAddress}}'  # 10.88.0.2
podman rm -f -t 0 s p
```

Question 3 — `ping` par IP, `ping` par nom

Réponse. Sur `podman` : le `ping 10.88.0.2` réussit (même bridge), le `ping a` échoue avec `bad address 'a'`. Sur `stack` : les deux réussissent. La différence est le champ `dns_enabled` de `podman network inspect` : `false` sur `podman`, `true` sur un réseau créé par vous — qui lance alors `aardvark-dns`.

Pourquoi. Le bridge assure la connectivité IP ; la résolution des noms est un service séparé, que Podman n'active que sur les réseaux user-defined (comme Docker).

Nuance. Même sur `stack`, `ping` par nom peut échouer dans une image qui n'a pas la permission `NET_RAW` ; `wget`, `nc` ou `getent hosts` sont des tests plus fiables qu'un `ping`.

Exemple.

```
podman network inspect podman --format '{{.DNSEnabled}}'   # false
podman network inspect stack  --format '{{.DNSEnabled}}'   # true
podman run --rm --network stack alpine getent hosts api    # 10.89.0.3  api.dns.podman …
```

Question 4 — « `EXPOSE` ne sert à rien »

Réponse. Il a tort sur le fond, raison sur le constat. `EXPOSE` **ne publie pas** de port : c'est une métadonnée de l'image qui dit « ce service écoute sur 8080 ». Elle est lue par `-P` (qui publie tous les ports exposés sur des ports hôte aléatoires), affichée par `podman ps` (`8080/tcp` sans flèche) et `podman image inspect` (`ExposedPorts`), et sert de documentation à qui déploie l'image.

Pourquoi. Publier est une décision de **déploiement** (quel port de quelle machine), pas de construction d'image. L'image ne peut pas la prendre.

Nuance. Entre conteneurs d'un même réseau, ni `EXPOSE` ni `-p` ne sont nécessaires : tous les ports du conteneur sont joignables. `EXPOSE` reste une bonne pratique : les outils (Compose, Kubernetes, les scanners) s'en servent.

Exemple.

```
podman image inspect api:1.0 --format '{{.Config.ExposedPorts}}'   # map[8080/tcp:{}]
podman run -d --name r --network stack -P api:1.0 && podman port r  # 8080/tcp -> 0.0.0.0:39201
podman rm -f -t 0 r
```

Question 5 — Port 80 refusé

Réponse. Avec Docker classique, c'est le **daemon root** qui ouvre le port 80 : aucune restriction. En rootless, c'est votre utilisateur, et le noyau réserve les ports < 1024 à root. Trois issues : (1) publier sur un port haut, `-p 8080:80`, et accepter l'URL avec `:8080` ; (2) abaisser la limite système, `sudo sysctl net.ipv4.ip_unprivileged_port_start=80` (persistant dans `/etc/sysctl.d/`), ce qui l'ouvre à tous les utilisateurs ; (3) placer devant un reverse proxy ou un équilibreur qui, lui, tourne en root ou en service système. En entreprise : (3) — un conteneur applicatif n'écoute jamais sur 80/443 directement.

Pourquoi. La restriction est une règle du noyau Linux, antérieure aux conteneurs : un port privilégié garantissait autrefois que le service était lancé par l'administrateur.

Nuance. Le conteneur est créé mais pas démarré : `podman ps -a` le montre en `Created`. Il faut le supprimer avant de relancer avec le même nom.

Exemple.

```
podman ps -a --filter name=web --format '{{.Status}}'   # Created
podman rm web && podman run -d --name web -p 8080:80 nginx:alpine
```

Question 6 — `502` après recréation de l'API

Réponse. `502 Bad Gateway`. nginx a résolu `api` **au démarrage** en `10.89.0.3` et garde cette adresse. La nouvelle `api` a reçu une autre IP (`autre` a pris la suivante), l'ancienne n'existe plus : `connect() failed (113: Host is unreachable)`. Le conteneur jetable, lui, résout `api` au moment de sa requête et trouve la bonne adresse. Réparation : `podman restart web` (nginx re-résout au démarrage), ou une configuration nginx avec `resolver 10.89.0.1 valid=10s;` et `set $upstream http://api:8080; proxy_pass $upstream;`, qui force une résolution à chaque requête.

Pourquoi. Par défaut, nginx résout les noms des `proxy_pass` statiques à la lecture de la configuration, une seule fois, pour ne pas payer une requête DNS par connexion.

Nuance. Tant qu'`api` est seulement **arrêtée**, le code est `504 Gateway Time-out` (l'IP ne répond plus) ; c'est la **recréation avec une autre IP** qui donne `502`. Spring Boot vers PostgreSQL n'a pas ce défaut : chaque nouvelle connexion du pool refait la résolution.

Exemple.

```
podman logs web 2>&1 | grep -i error | tail -1     # upstream: "http://10.89.0.3:8080/…"
podman inspect api --format '{{.NetworkSettings.Networks.stack.IPAddress}}'   # 10.89.0.8
podman restart -t 1 web && curl -s -o /dev/null -w '%{http_code}\n'
localhost:18080/api/actuator/health   # 200
```

Question 7 — L'adresse client, c'est le conteneur

Réponse. En rootless, le port publié est tenu par `rootlessport`, un relais en espace utilisateur qui **accepte la connexion sur l'hôte et en ouvre une nouvelle** vers le conteneur, depuis l'intérieur du namespace réseau de Podman : nginx voit donc comme client l'adresse du conteneur lui-même. Avec Docker root, le port est relayé par des règles de NAT du noyau qui conservent l'adresse source réelle. Conséquence : toute logique basée sur l'IP cliente (journalisation, limitation de débit, liste blanche) est aveugle derrière un port publié rootless.

Pourquoi. Un relais applicatif (proxy TCP) termine la connexion, là où le NAT ne fait que réécrire des en-têtes de paquets.

Nuance. Sur le réseau par défaut (`pasta`), l'adresse source est mieux préservée. Et en production, le problème se résout par l'en-tête `X-Forwarded-For` ajouté par l'équilibreur ou le reverse proxy frontal.

Exemple.

```
podman logs web 2>&1 | grep GET | tail -1 | cut -d' ' -f1   # 10.89.0.4
podman inspect web --format '{{.NetworkSettings.Networks.stack.IPAddress}}'   # 10.89.0.4
```

Question 8 — Deux réseaux

Réponse. Oui. Cas typique : un réseau `admin` pour les outils de supervision ou de sauvegarde, auquel on connecte `api` (ou `db`) sans y mettre `web`. Commandes : `podman network create admin`, `podman network connect --alias backend admin api`, `podman network disconnect admin api` pour défaire. L'inspect montre alors deux clés, `stack` et `admin`, chacune avec sa propre `IPAddress` (`10.89.0.3` et `10.89.1.3`) et ses alias.

Pourquoi. Chaque réseau est un bridge distinct ; connecter un conteneur ajoute une interface (`eth1`) dans son namespace. Le DNS de chaque réseau ne connaît que ses membres.

Nuance. Un conteneur de `admin` ne voit **pas** `web` ni `db` : la segmentation est réelle. C'est un outil d'isolation plus qu'une commodité.

Exemple.

```
podman network connect --alias backend admin api
podman inspect api --format '{{range $k,$v := .NetworkSettings.Networks}}{{$k}}=
{{$v.IPAddress}} {{end}}'   # admin=10.89.1.3 stack=10.89.0.3
podman run --rm --network admin alpine wget -qO- http://backend:8080/actuator/health
```

Question 9 — `--network host`

Réponse. (1) **Plus de `-p`** : le conteneur écoute directement sur les ports de l'hôte, donc deux API en `--network host` sur 8080 entrent en conflit. (2) **Plus de DNS interne ni d'isolation** : `api` ne résout plus `db` par son nom, et le conteneur voit toutes les interfaces et tous les ports de l'hôte. Légitime pour un outil réseau (`tcpdump`, scanner, agent de supervision) ou pour des performances réseau extrêmes. En rootless, une application qui veut le port 80 échoue, comme n'importe quel processus de votre utilisateur : `bind() to 0.0.0.0:80 failed (13: Permission denied)`.

Pourquoi. `--network host` ne crée pas de network namespace : le processus partage celui de l'hôte, avec les mêmes contraintes et les mêmes conflits.

Nuance. Ce que le conteneur voit en `--network host` rootless, c'est le namespace de **votre utilisateur** sur l'hôte, pas celui du daemon : identique en pratique à un processus lancé sans conteneur.

Exemple.

```
podman run --rm --network host alpine ip -4 -o addr show | grep -v ' lo '   # vos interfaces
WSL
podman run --rm --network host nginx:alpine sh -c 'nginx; sleep 1'          # bind() …
Permission denied
```

Question 10 — Réseau ou pod

Réponse. Dans le réseau `stack`, nginx écrit `proxy_pass http://api:8080/` (nom résolu par le DNS), et `-p 18080:80` se met sur `web`. Dans le pod `stackpod`, nginx écrit `proxy_pass http://localhost:8080/` (même namespace), et `-p 18090:80` se déclare **à la création du pod** ; `podman ps --pod` montre une colonne `POD ID` et un conteneur `…-infra` qui porte la publication. Ce second modèle est celui de **Kubernetes** : un Pod y est un groupe de conteneurs partageant réseau et `localhost`, et les manifestes de la partie 2 de cette série reposent dessus.

Pourquoi. Le pod partage un seul network namespace entre ses membres ; le réseau relie des namespaces séparés.

Nuance. Dans un pod, deux conteneurs ne peuvent pas écouter sur le même port : un seul `8080` par pod. Et `/etc/hosts` du pod liste tous ses conteneurs sur `127.0.0.1`.

Exemple.

```
podman pod create --name stackpod -p 18090:80
podman run -d --pod stackpod --name api-pod api:1.0
podman exec api-pod cat /etc/hosts | tail -2    # 127.0.0.1 api-pod …
```

Question 11 — DBeaver sur `db`

Réponse. `-p 127.0.0.1:5432:5432` : le port n'est ouvert que sur l'interface de bouclage de WSL. Le navigateur ou DBeaver sous Windows l'atteint quand même parce que WSL 2 **relaie vers `localhost` côté Windows** tout port écouté dans la distribution, y compris ceux liés à `127.0.0.1`. Les autres machines du réseau, elles, ne voient rien. En production, on ne publie jamais la base : elle n'est joignable que par les services de son réseau, et les humains passent par un bastion ou un outil d'administration dédié.

Pourquoi. Publier un port, c'est ouvrir une porte sur la machine ; `127.0.0.1:` la réserve aux processus locaux. Le relais WSL est une commodité de Windows, pas une règle de Podman.

Nuance. En mode réseau *mirrored* de WSL, les interfaces Windows et WSL sont partagées et un `-p 0.0.0.0:5432:5432` exposerait la base au réseau local. Gardez `127.0.0.1:` par réflexe.

Exemple.

```
podman run -d --name db --network stack -p 127.0.0.1:5432:5432 -e POSTGRES_PASSWORD=secret
postgres:16-alpine
podman port db   # 5432/tcp -> 127.0.0.1:5432
```

Question 12 — `network rm` refusé

Réponse. `-f` supprime d'abord **tous les conteneurs et pods** connectés au réseau, puis le réseau. Dans un nettoyage, c'est une mauvaise idée parce qu'on ne lit pas la liste de ce qui va disparaître — et qu'un conteneur peut appartenir à ce réseau sans qu'on l'ait en tête (un outil d'administration, une base de test). Séquence correcte : `podman network inspect stack --format '{{range .Containers}}{{.Name}} {{end}}'` pour voir, `podman rm -f -t 0 web api db` nommément, puis `podman network rm stack`.

Pourquoi. Un réseau avec des membres est une ressource en cours d'utilisation ; Podman protège par défaut, comme pour `volume rm`.

Nuance. Un conteneur **arrêté** compte toujours comme membre : la suppression échoue même si rien ne tourne.

Exemple.

```
podman network inspect stack --format '{{range .Containers}}{{.Name}} {{end}}'   # api db web
podman rm -f -t 0 web api db && podman network rm stack
```

Question 13 — Reverse proxy contre CORS

Réponse. Le navigateur applique le CORS quand une page appelle une **autre origine** (schéma + hôte + port). Si la page vient de `localhost:18080` et l'API de `localhost:8080`, les ports diffèrent : autre origine, en-têtes CORS obligatoires. Avec nginx qui sert `/` et relaie `/api/` vers `api:8080`, le navigateur ne parle qu'à `localhost:18080` : même origine, pas de CORS. Autres bénéfices : l'API n'est plus publiée (surface réduite), un seul point pour le TLS, la compression, le cache et les logs,

et la même URL `/api/` en développement et en production. La ligne qui fait le travail : `proxy_pass http://api:8080/;` dans un bloc `location /api/`.

Pourquoi. Le CORS est une politique du **navigateur**, pas du serveur : un proxy côté serveur, qui n'est pas un navigateur, n'y est pas soumis.

Nuance. Le `/` final de `proxy_pass http://api:8080/` retire le préfixe `/api/` avant de transmettre : `/api/actuator/health` devient `/actuator/health`. Sans ce `/`, l'API recevrait `/api/actuator/health` et répondrait `404`.

Exemple.

```
curl -s localhost:18080/api/actuator/health     # {"status":"UP"} — via nginx, sans en-tête CORS
podman logs web 2>&1 | grep 'GET /api/' | tail -1
```